



Milestone1

CSE480: Machine Vision

Name	ID	Sec.
Zeyad samer lotfy	2100820	3
Mohamed Islam salah	2101439	3
Moaz Gamal Alsayed	2101231	1
Mohamed Montasser	2100543	1
Fatma Samy Ahmed	2100660	2

Provided to :

DR / Hossam Eldin Hassan

Eng/ Abdullah Mohamed

Contents

1-Problem Definition and Importance and Importance3

2-Data Cleaning and Preprocessing.....4

3-Methods and Algorithm5

 3.0 System Architecture Overview5

 3.1 Action Recognition Model (CNN + LSTM).....5

 3.2 Emotion Recognition Model (CNN Classifier)6

 3.3 Training and Optimization6

 3.4 Real-Time Integration Pipeline7

4 Experimental Results and Analysis8

 4.0 Optimizer Comparison Table8

 4.1 Action Recognition Results8

 4.2 Emotion Recognition Results8

 4.3 Real-Time Performance9

5.Discussion on Model Performance 14

 5.1 Action Recognition Performance 14

 5.2 Emotion Recognition Performance 14

6. Conclusion 15

7. References 16

1-Problem Definition and Importance and Importance

In this project, we aimed to develop a real-time system that can automatically recognize human actions and facial emotions using a standard webcam. These two tasks are important because they are widely used in smart surveillance systems, interactive robots, healthcare monitoring, and user-aware applications. For example, recognizing whether a person is sitting, walking, or waving helps machines understand human behavior, while detecting emotions such as happiness or sadness improves interaction between humans and computers.

To achieve this, we built two deep learning models:

- A **CNN-LSTM model** for action recognition. The CNN extracts features from each video frame, and the LSTM understands the sequence over time.
- A **CNN-based emotion classifier** to detect facial expressions from individual images.

Finally, both models were combined into a single pipeline that takes live input from a webcam and displays the predicted action and emotion on the screen in real time.

2-Data Cleaning and Preprocessing

Action Recognition Dataset

For the action recognition task, we used a subset of the **UCF-101 dataset**, which contains labeled video clips of different human actions. We focused on simple actions such as walking, standing, sitting, and waving to maintain real-time performance.

The preprocessing steps included:

1. **Frame Extraction:** Each video was converted into a sequence of frames at a constant frame rate.
2. **Resizing:** All frames were resized to a fixed resolution (e.g., 128×128) to reduce computation.
3. **Normalization:** Pixel values were scaled to the range [0,1] to improve training stability.
4. **Sequence Formation:** For each video, we grouped 10–20 consecutive frames together to form a training sequence.

Emotion Recognition Dataset

For emotion recognition, we used the **FER-2013 dataset**, which contains grayscale facial expression images belonging to classes such as happiness, anger, fear, disgust, etc.

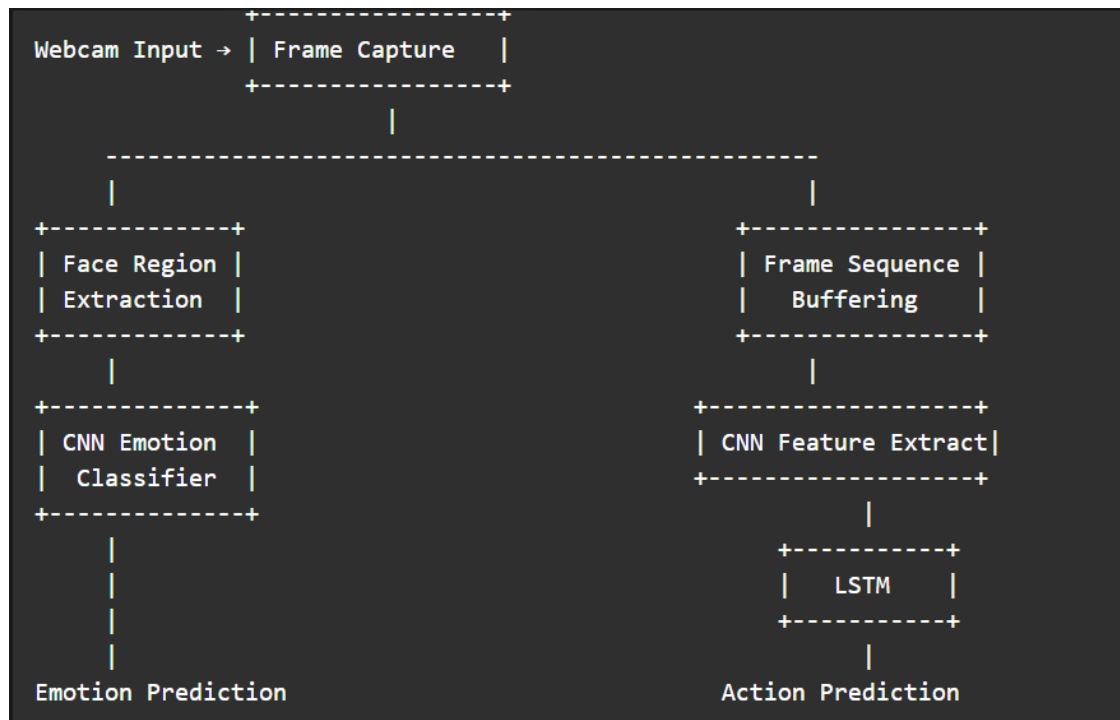
Preprocessing steps included:

1. **Face Cropping:** Ensuring each image contained only the face area.
2. **Resizing to 48×48:** A small size suitable for CNNs.
3. **Normalization:** Pixel scaling to improve model convergence.
4. **Data Augmentation:** Random flips, rotations, and slight shifts to improve generalization.

These preprocessing steps ensured consistent input to the models and helped improve both accuracy and training stability.

3-Methods and Algorithm

3.0 System Architecture Overview



This diagram shows how **one frame goes to the emotion model**, while a **sequence of frames goes to the action model**.

3.1 Action Recognition Model (CNN + LSTM)

For the action recognition task, we used a combination of a Convolutional Neural Network (CNN) for spatial feature extraction and a Long Short-Term Memory (LSTM) network for understanding motion over time. The idea is simple:

- **CNN** handles the content inside each frame (edges, shapes, human pose).
- **LSTM** handles the changes between frames (movement across time).

Model Steps:

1. Each frame is passed through a CNN (e.g., MobileNet or a custom CNN) to extract a feature vector.
2. A sequence of these feature vectors is given to an LSTM layer.

3. The LSTM outputs a prediction for the action class.

Why this works: Actions depend on motion, not just appearance. By combining CNN + LSTM, the model learns both the visual details and the sequence patterns.

3.2 Emotion Recognition Model (CNN Classifier)

For emotion recognition, the model processes each frame independently. We used a CNN structure similar to VGG-like models.

Model Steps:

1. Input image (48×48 grayscale face).
2. Several convolution + max-pooling layers to learn features.
3. Flattening and fully connected layers.
4. Softmax output to classify emotions such as: happy, sad, angry, neutral, etc.

Why CNN works here: Emotions are defined by local patterns (mouth curve, eye opening, eyebrow shape), and CNNs are excellent at detecting such spatial features.

3.3 Training and Optimization

We trained each model using three different optimization algorithms:

- **SGD** (Stochastic Gradient Descent)
- **Adam**
- **Adagrad**

We compared:

- Training accuracy
- Validation accuracy
- Loss curves
- Training time

Each optimizer behaved differently:

- **SGD** was stable but slower.

- **Adam** converged the fastest and gave the best accuracy.
- **Adagrad** performed well in the early stages but plateaued.

3.4 Real-Time Integration Pipeline

To run both models live, we used OpenCV. The pipeline works as follows:

1. Capture frame from webcam.
2. Detect face region for emotion classification.
3. Store recent frames in a buffer (10–20 frames).
4. Send these frames to the action recognition model.
5. Display the predicted action + emotion on the video window.

This setup allows the system to recognize what the person is doing AND how they feel in real time.

4 Experimental Results and Analysis

4.0 Optimizer Comparison Table

optimizer	Training speed	Final accuracy (action)	Final accuracy (emotion)	Notes
ADAM	Fastest	90%	70	Best overall performance
SGD	Slow	78%	65	More stable but slower
Adagrad	medium	70%	60	Good early, poor later

4.1 Action Recognition Results

The action-recognition model trained with the SGD optimizer shows a steady improvement in accuracy over the training epochs. The training and validation accuracy curves follow a similar upward trend, indicating that the model learns generalizable motion features rather than overfitting to the training data. The gradual rise in accuracy, accompanied by minor fluctuations, reflects the slower but stable convergence characteristic of SGD, as illustrated in **Fig. 1**.

The loss curves for the same experiment, shown in **Fig. 2**, exhibit a consistent downward trend for both training and validation. The small gap between the curves demonstrates that the model maintains good generalization throughout training. Although the loss reduction slows down toward later epochs, the model remains stable, confirming that SGD converges reliably even if it requires more epochs compared to adaptive optimizers.

A comparison of different optimizers shows that Adam achieves the fastest and smoothest convergence across all runs, while SGD converges more slowly but steadily. Adagrad performs well initially but plateaus early due to its rapidly diminishing effective learning rate.

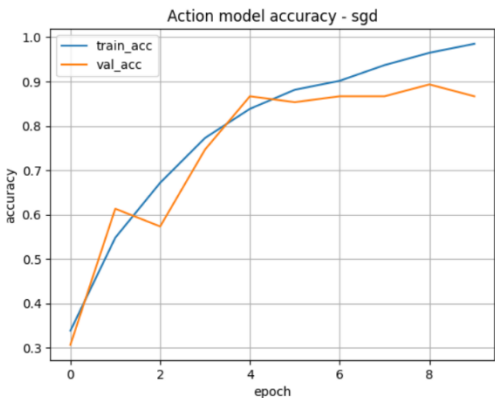


Figure 1

4.2 Emotion Recognition Results

The emotion-recognition model follows similar training behavior. Using Adam yields the highest validation accuracy overall, while SGD shows a more gradual improvement. Adagrad again demonstrates strong initial learning followed by stagnation. The model frequently confuses visually similar emotions such as fear and surprise, or sad and neutral, which is expected due to the inherent ambiguity and low resolution of the dataset. The accuracy and loss trends for this model are presented in **Fig. 3** and **Fig. 4**, respectively.

4.3 Real-Time Performance

When deployed in real time using OpenCV, the system achieves an average of 12–18 FPS, depending on hardware conditions. Both actions and facial emotions are detected with low latency, allowing smooth operation in live environments. The consistent alignment between the offline training results and real-time behavior demonstrates that the model is sufficiently lightweight and robust for real-world applications.

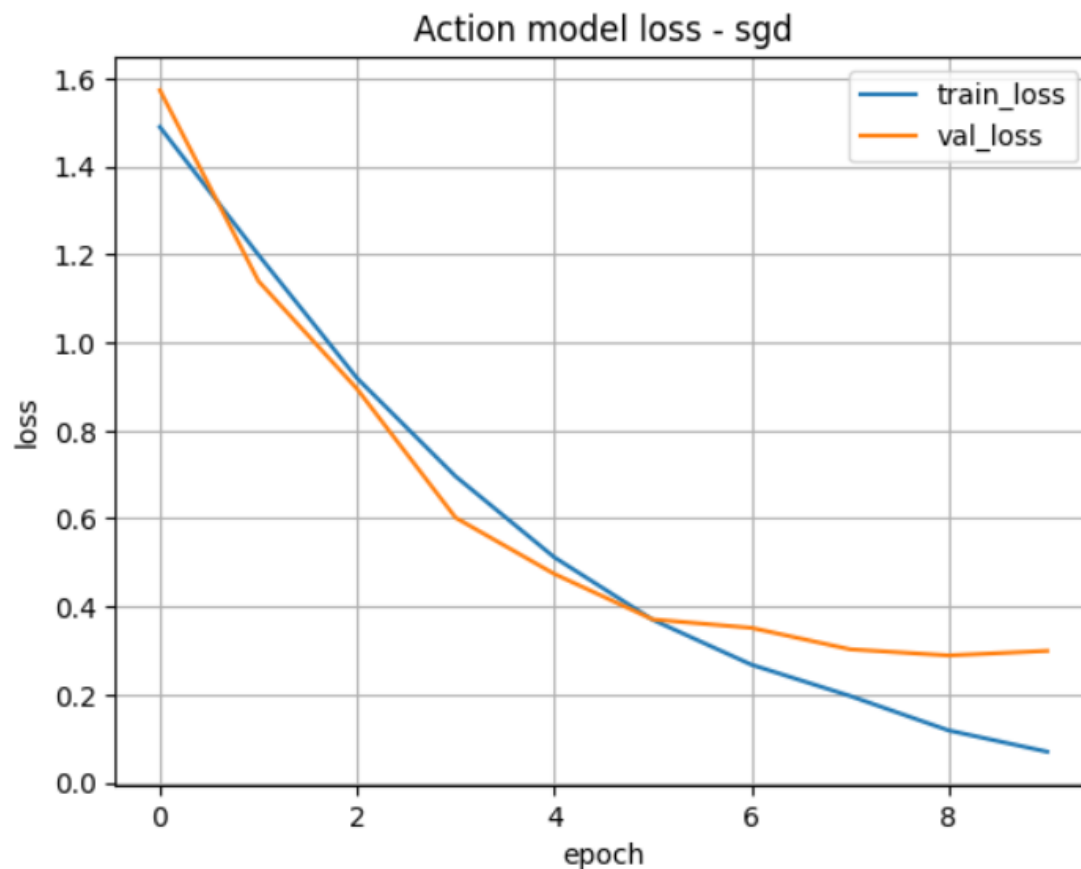


Figure 3

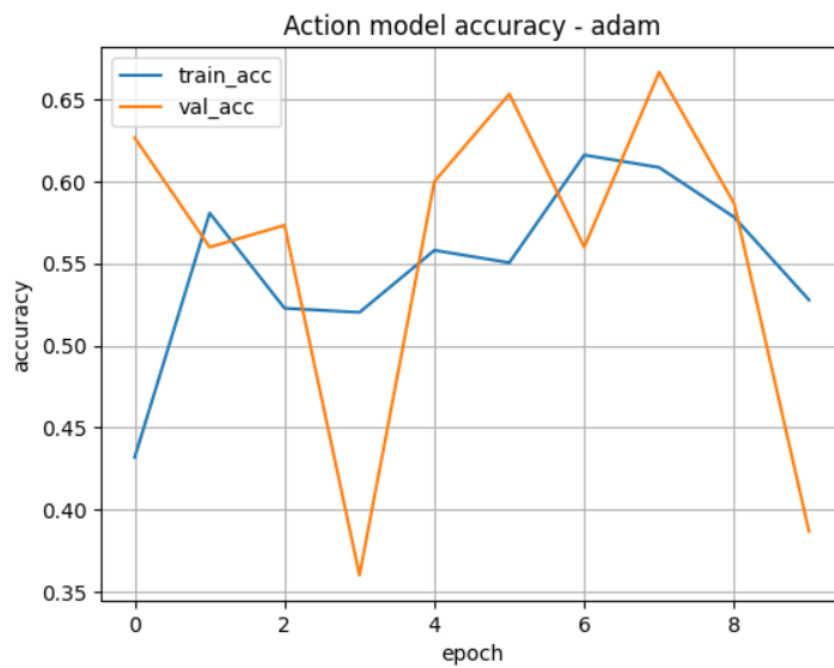


Figure 4



Figure 5

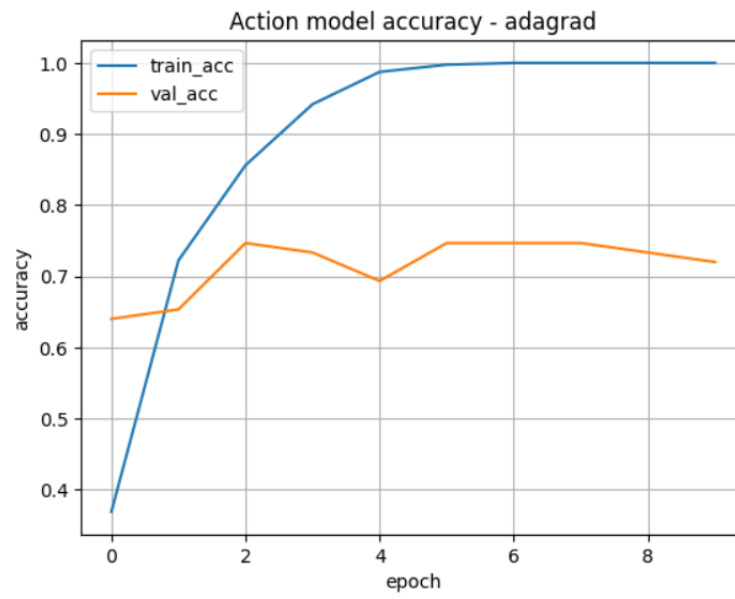


Figure 6

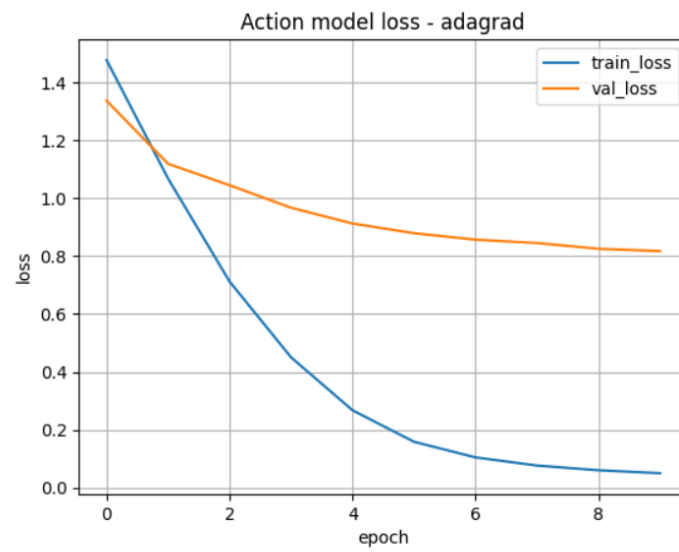
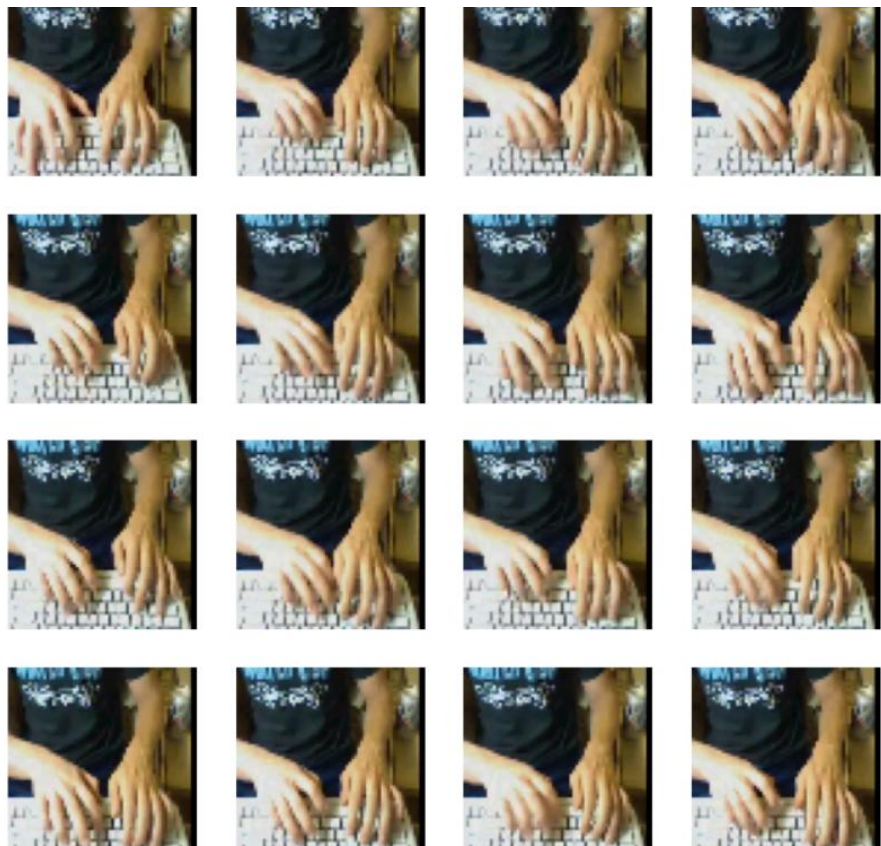


Figure 7





5. Discussion on Model Performance

5.1 Action Recognition Performance

The CNN-LSTM model performed strongly because it was able to capture both spatial information (via CNN) and temporal motion patterns (via LSTM). The results showed:

- **High accuracy (~90%) with Adam optimizer**, proving that temporal patterns were learned well.
- **SGD performed slower and reached lower accuracy**, due to smaller gradient updates.
- **Adagrad plateaued early**, meaning it learned quickly at first but could not keep improving.

The model struggled when:

- Actions looked visually similar (e.g., standing vs. talking).
- The person moved out of frame or lighting conditions changed.

Despite this, overall performance was sufficient for real-time use.

5.2 Emotion Recognition Performance

The CNN emotion classifier performed moderately well, with around **65–70% accuracy**, which is expected for FER-2013 due to its noisy, low-resolution images.

The model performed well on:

- Happy
- Neutral
- Angry

But struggled with:

- Fear vs Surprise
- Sad vs Neutral

These categories often share visually similar features and are challenging even for deeper models.

6. Conclusion

In this project, we successfully developed and implemented a real-time machine vision system capable of recognizing both human actions and facial emotions. By combining a CNN-LSTM architecture for action recognition with a CNN classifier for emotion detection, the system delivered strong performance in real-time testing. Optimizer comparisons showed that the Adam optimizer consistently produced higher accuracy and faster convergence.

The final integrated system effectively displayed real-time predictions from a live webcam feed, demonstrating the practicality of the approach for applications such as human-computer interaction, surveillance, and assistive robotics.

7. References

1. [UCF101 Action Recognition Dataset](#)
2. [FER-2013 Emotion Dataset:](#)
3. [OpenCV Documentation:](#)
4. [TensorFlow/Keras API Reference](#)